

```
def split(s):
    out = []
    term = ''
    for c in s:
        if c == ' ':
            out.append(term)
            term = ''
        else:
            term += c

    # add the final word unless it is empty
    if term != '':
        out.append(term)

    return out
```

Funktionen ska funka så här:

```
print(annotate_word('hej du hej hej du'))
```

```
{'hej': 4, 'du': 5}
```

9. Hur kan vi enkelt få `annotate_word` i uppgift 8 att istället returnera en uppslagstabell med tal som nycklar och ord som värden?
10. Modifiera `annotate_word` i uppgift 8 så att inte bara sista förekomsten sparas i texten, utan alla förekomster. Vilken datastruktur bör vi ha som *värde* i vår uppslagstabell?

Kapitel 5

Felhantering, sårfall, listomfattning, och felsökning

Det blir fel ibland, även när man kör program, och felen måste då hanteras. När man programmerar är det typiskt status på en funktion man är intresserad av: gick beräkningen bra eller har det uppstått ett fel? Hur fortsätter vi efter felet?

Hittills i kompendiet har vi definierat funktioner som returnerar resultat (värden), samt att något instruktivt meddelande kanske har skrivits ut med hjälp av att anropa `print`. Alternativt har programmet bara kraschat eller fastnat i en oändlig loop.

I detta kapitel kommer vi att undersöka hur osäker kod vid fel kan påkalla, eller "lyfta", ett **sårfall**. Detta kan hanteras på ett lämpligt sätt med hjälp av **felhantering** vilket i Python görs med **try** och **except**.

Vi ska även i slutet av kapitlet titta på ett helt annat koncept: listomfattningar. Detta är ett smidigt sätt att skapa och modifiera listor, som ofta är mycket mer kompakt än att skriva en loop.

5.1 Felhantering och sårfall

Ett naivt sätt att hantera fel i program är att returnera en felkod i form av ett tal eller en sträng. Vi ska nu börja med att titta på hur detta kan se ut, men sen snabbt gå över till att titta på hur fel kan hanteras bättre med hjälp av sårfall.

5.1.1 Felhantering med hjälp av returnerade felkoder

Att förlita sig på returvärden för felhantering ger komplexa program som tenderar att bli svårlästa. Många funktionsanrop måste följas av **if**-satser för att se om ett fel uppstod som man måste anpassa sig efter. Om ett fel uppstod kan det bli knepigt att återgå till en bra plats i programmet. Erfarenheten är att programmerare undviker den ökade komplexiteten i sitt program genom att ignorera de returnerade felkoderna vilket ger instabila program som lätt kraschar. Denna typ av felhantering förekommer ofta i t.ex. C, Fortran och andra äldre programmeringsspråk.

Ett exempel på denna gamla teknik följer nedan där vi anropar en funktion som ska modifiera en lista. Alla returkoder förutom 0 signalerar att något fel har skett.

```
def divide_list(l,x):
    if type(x) != int or type(x) != float:
        return 11
    elif x == 0:
        return -1
    else:
        # check that elements in list are integers
        if not all([ type(item) == int for item in l]):
            return 12
        for i in range(len(l)):
            l[i] = l[i]/x # we are modifying the list items here
        return 0

return_code = divide_list(l,x)
if return_code == -1:
    # do something
elif return_code == 11:
    # do something else...
elif return_code == 12:
    # do something else...
elif return_code == 0:
    # good!
```

Denna kod är ganska rörig och onödigt komplicerad. Många moderna språk stödjer därför specifika sätt att hantera fel på genom så kallade "särfall" (eng: *exceptions*, ibland översatt till "undantag").

5.1.2 Särfall

Python använder sig av de reserverade orden **try** och **except** för att dela in koden i block som vi inte litar på, vi kan kalla dem osäkra passager, och block som exekveras vid ett särfall med syfte att hantera problemet. Här är grundstrukturen:

```
try:
    # unsafe code
    ...
except:
    # action
    ...

# Rest of the code
...
```

Konstruktionen **try** lägger man alltså runt ett kodblock som kan betraktas som en osäker passage (# *unsafe code* ovan) och efter **except** lägger man de åtgärder som man hanterar eventuella problem med. Indenteringen är viktig här, precis som vid funktionsdefinitioner, villkorssatser och kontrollflöden. Blocket som hör till **except**-satsen (# *action*) exekveras endast om ett fel har gett ett särfall i den osäkra passagen under **try**-satsen: om inget fel har uppstått fortsätter koden utan att **except**-blocket körs. Om ett fel uppstår, så kan åtgärderna under **except** antingen avsluta exekveringen av programmet eller så kan felet behandlas på något annat sätt (t.ex. ignoreras, även om man ska vara försiktig med att strunta i fel som ger ett särfall).

5.1.2.1 Exempel: läsa in ett heltal från användaren

Betrakta följande kodsnitt:

```
while True:
    answer = input("Write a number (write 0 to quit): ")
    x = int(answer)
    if x == 0:
        print("Bye bye!")
        break
    else:
        print("The square of the number is: " + str(x ** 2))
```

Vad händer om användaren skriver in något annat än ett tal?

```
Write a number (write 0 to quit): hej
```

```
ValueError: invalid literal for int() with base 10: 'hej'
```

Detta är, som vi sett tidigare, en konsekvens av att resultatet av `input` alltid har typen `str`, och att det inte alltid går att explicit typomvandla från `str` till `int`. Här är `ValueError` ett särfall: värdet av inparametern ('hej') till `int` är felaktigt. Om man istället använder sig av `try/except`-sats så kan felet hanteras på rätt sätt:

```
while True:
    try:
        answer = input("Write a number (write 0 to quit): ")
        x = int(answer)
        if x == 0:
            print("Bye bye!")
            break
        else:
            print("The square of the number is: " + str(x ** 2))
    except ValueError:
        print("You must write a number!")
```

I koden ovan fångas alla särfall av typ `ValueError` som lyfts i `try`-blocket. Om ett annat typ av särfall skulle påträffas skulle programmet avslutas. Man kan även fånga *alla* fel med endast `except`:

```
while True:
    try:
        answer = input("Write a number (write 0 to quit): ")
        x = int(answer)
        if x == 0:
            print("Bye bye!")
            break
        else:
            print("The square of the number is: " + str(x ** 2))
    except:
        print("You must write a number!")
```

För att koden ska bli så överskådlig och tydlig som möjligt, vill man dock helst skriva så lite kod som möjligt innanför `try`-blocket. Genom att vara specifik i sin avsikt kan exemplet ovan förbättras till

följande:

```
while True:
    answer = input("Write a number (write 0 to quit): ")
    try:
        x = int(answer)
    except ValueError:
        print("You must write a number!")
        continue

    if x == 0:
        print("Bye bye!")
        break
    else:
        print("The square of the number is: " + str(x ** 2))
```

Med continue ser vi till att starta om while-loopen, innan x börjar användas. Vi har identifierat att det bara är typkonverteringen som kan orsaka problem.

5.1.2.2 Exempel: beräkna längden av rader i en fil

Låt oss säga att vi vill beräkna längden på alla rader i en fil:

```
def len_file(filename):
    with open(filename, 'r') as h:
        for s in h:
            print(len(s))

len_file("foo.txt")
```

Om filen foo.txt inte finns får vi följande

```
FileNotFoundError: [Errno 2] No such file or directory: 'foo.txt'
```

Det är bättre att skriva:

```
def len_file(filename):
    try:
        with open(filename, 'r') as h:
            for s in h:
                print(len(s))
    except FileNotFoundError:
        print("Could not open file " + filename + " for reading.")

len_file("foo.txt")
```

Fördelen är att vi får ett program som kommunicerar med ett mer lättläst felmeddelande till användare. Det "vanliga" sättet som Python signalerar fel med är mer till för programmerare. Genom att hantera fel som kan uppstå vid till exempel felhantering kan man göra sitt program mer användarvänligt.

5.1.2.3 Olika åtgärder för specifika sårfall

Det är mycket som kan gå fel och olika sårfall kan man vilja hantera på olika sätt. I nästa exempel så görs separata åtgärder för om något går fel vid filinläsningen, om ett tal delas på noll, eller om den inlästa strängen från filen inte kan typomvandlas till ett heltal. Notera att åtgärden inte behöver placeras nära platsen där felet uppstår.

```
def divide_by_elems(filename,x):
    quotients = []
    with open(filename, 'r') as h:
        for n in h:
            frac = x / int(n)
            quotients.append(frac)
    return quotients

try:
    data = divide_by_elems('numbers.txt', 2)
except IOError:
    print("divide_by_elems: A file-related problem occurred.")
except ZeroDivisionError:
    print("divide_by_elems: Division by zero.")
except ValueError:
    print("divide_by_elems: Character could not be converted to int.")

print(data)
```

Om koden ovan skulle köras med en fil `numbers.txt` som innehåller följande:

```
1
2
0
4
3
0
```

så genereras och fångas ett sårfall. Vilket? Kan du provocera fram de två andra sårfallen?

5.1.2.4 Sårfall för kontrollflöde

Man kan även använda sårfall för kontrollflöde i sitt program. Låt oss säga att vi vill beräkna kvoten mellan varje element i två listor (och ignorera alla element i slutet av den längre listan), men inte krascha om det blir division med noll. Då kan vi skriva följande:

```
def compute_ratios(xs,ys):
    ratios = []
    for i in range(min(len(xs),len(ys))):
        try:
            ratios.append(xs[i]/ys[i])
        except ZeroDivisionError:
            ratios.append(float('NaN')) # NaN = Not a Number
    return ratios
```

```
print(compute_ratios([2,3,4],[2,0,5]))
print(compute_ratios([2,3,4,0],[2,0,5]))
```

I detta fall hade vi dock lika gärna kunnat använda en **if**-sats och testat om `ys[i] == 0`. Mer om detta senare.

5.1.3 Allmänt om särfall

Det anses vara idiomatisk Python att använda sig av **try-except**. Det är inte ovanligt att när man läser om Python-kod ser *EAFP*, som står för *It's Easier to Ask Forgiveness than Permission*. Det här kontrasteras mot *Look Before You Leap (LBYL)*. Det är inte menat att man ska överanvända **try-except**: särfall som uppstår kan ibland ge den tydligaste informationen. Varje möjliga särfall i kod är inte menat att behandlas. Men osäkra passager där man har en tydlig åtgärd i åtanke är ett bra användningsområde.

Så vad är en osäker passage i ett program? Typiskt är:

- All filhantering.
- Kommunikation med användare.
- Kod som annonserar osäkerhet i dokumentationen ("throws exception if...").
- Kod som man vet eller upptäcker är osäker (t.ex. om man implementerar en algoritm som är instabil).

Det finns flera vanliga åtgärder när man fångat ett särfall:

- **Skriv ut felmeddelande och avsluta:** det gör man till exempel om data saknas eller om felet är så allvarligt att det inte finns rimliga sätt att fortsätta.
- **Skriv ut varning, ignorera felet, och fortsätt:** om man har konstiga indata som man kan bortse ifrån, och beräkningarna kan fortsätta ändå.
- **Hantera felet och fortsätt:** fungerar bra för en del förutsägbara fel, om man t.ex. får en dålig inmatning från en användare kan man ju be om ett nytt försök.
- **Skapa ett nytt särfall:** det är lämpligt om du har kod där flera fel kan uppstå, men där felet kan sammanfattas bättre i ett visst sammanhang. Om en algoritm är instabil så kanske `ArithmeticError` är ett bättre fel än `ZeroDivisionError`, eftersom användaren av algoritmen känner till algoritmens svaghet men kanske inte kan associera det med division med noll.

Det finns också åtgärder man ska undvika:

- **Ignorera inte felet!** Det värsta man kan ha efter **except** är instruktionen **pass**. Som användare blir det väldigt svårt att veta vad och var någonting går fel. Python har lyft ett särfall, men man har som programmerare bestämt att strunta i detta. Resultatet blir ofta att man får konstiga följdfel på en helt annan plats i koden.
- För **allvarliga** fel *räcker inte* en felutskrift. Det är inte snyggt att låta ett program fortsätta om felet är så allvarligt att man vet att det kommer uppstå ett annat fel senare. Vissa fel ska helt enkelt avbryta ett program.

5.1.4 Hantera egna stabilitetsproblem

Det är inte nödvändigtvis en svaghet att problem uppstår. Det är svårt, om inte helt omöjligt, att t.ex.:

- Förutse formatet på *all* möjlig indata.
- Se till att *alla* användare (inklusive du själv) anropar funktioner helt rätt.

I Python finns många specifika särfall definierade (se [dokumentationen](#) för en detaljerad beskrivning):

- `Exception`
- `ArithmeticError`
- `IOError`
- `IndexError`
- `MemoryError`
- `ZeroDivisionError`
- m.m.fl.

I Python är sårfall inte bara en textstrång eller felkod som skrivs, sårfallet är en klass (klasser kommer att behandlas i mer detalj senare i kompendiet). Det gör det möjligt att lätt skapa egna sårfallsvarianter, som är baserade på de existerande sårfallen.

Man kan även lyfta ett generellt sårfall med det reserverade ordet **raise** (det är härifrån termen “lyfta” kommer). I t.ex. Java och C++, åstadkoms detta med hjälp av `throw`. Om vi t.ex. vill ta ut första elementet ur en lista och kasta ett fel om listan är tom kan vi skriva:

```
def head(xs):
    if not xs:
        raise Exception("head: input list is empty")
    else:
        return xs[0]

try:
    print(head([]))
except Exception as e:
    print(str(e))

print(head([]))
```

```
head: input list is empty
# specific information about where the exception was caught
Exception: head: input list is empty
```

Notera att koden fortsätter efter *första* gången `head` anropades i **try-except**-blocket: endast *strängen* med felmeddelandet skrivs ut. Man kan “spara” sårfallet i en variabel som man vidare kan använda (för t.ex. information, eller om man vill lyfta sårfallet *efter* man utfört en viss operation som att spara något till en fil) genom **except Exception as e**, där `e` är identifieraren för det fångade sårfallet. Strängen med felmeddelandet fås genom `str(e)`. *Andra* gången `head` anropas så lyfts sårfallet som stöts på i funktionen: samma information skrivs ut igen men man ser att det är **Exception** som sköter sårfallet snarare än **except**-blocket och programmet *avslutas*.

Nyckelordet **raise** är användbart för att skriva mer detaljerade felmeddelanden, eller när ett “fel” uppstår som inte täcks av Pythons felhantering. Ett exempel på ett sådant fel är om en algoritm som vi skrivit måste bibehålla ett värde på en parameter `h` större än `0.01` om algoritmen ska fungera (tänk t.ex. steglängd på `x`-axeln för att hitta nollställe till polynom). Vi kan då skriva något i stil med

```
def my_algorithm(x, h):
    while True:
        if h < 0.01:
            raise ArithmeticError('Stepsize h should not be smaller than 0.01.\
The value of h was:', h)
```



```
# check polynomial root
...

# update h
...
```

5.1.5 Vad du skriver påverkar hur du jobbar med särfall

Man kan säga att det finns olika scenarier för användning av särfall.

- **Applikationsprogrammering:** Om du skriver ett program som använder kod från olika moduler, såväl egna och som andras, så kommer det finnas funktioner och metoder som genererar särfall. Det blir då viktigt att använda **try-except** för att ta om hand om de särfall som kan uppstå. Programmet ska inte avslutas på grund av problem som kunnat förutses och som det kanske finns enkla lösningar till. Man kan också vilja undvika att användaren av ditt program (du själv eller andra) ska behöva se och tolka felmeddelanden.

Några exempel:

- Om ett särfall uppstår på grund av dålig indata, då ska man tala om det på ett tydligt sätt. Att släppa igenom ett felmeddelande från ett särfall är inte användarvänligt, eftersom den egentliga orsaken ofta inte alls förklarar varför särfallet uppstått.
- Om ett särfall uppstår på grund av, exempelvis, numerisk instabilitet eller osäkerheter i omgivningen (som att en fil med ett visst namn redan finns) så vill man antagligen hantera det på ett bra sätt istället för att slänga ett särfall i ansiktet på användaren.
- **Stödprogrammering:** Det är mycket vanligt att man skriver kod som är stöd till det program man verkligen vill skriva. Om ditt mål är att skriva en egen variant av Matlab så behöver du många funktioner som gör beräkningar; om du vill analysera data lagrade i ett komplicerat format kanske det är bra att ha en modul som just hanterar data så att du senare kan fokusera på beräkningarna. Dessa stödrutiner kan stöta på problem som måste signaleras till användaren och det görs lämpligen genom att använda **raise** till att skapa ett särfall.

Python talar om *att* ett fel uppstått. Du som programmerare kan tala om *varför* ett fel har uppstått.

5.1.6 Vanliga misstag

5.1.6.1 if-sats istället för särfall för flödeskontroll

Det kan vara lockande att använda **except** för flödeskontroll, men ofta är det bättre att istället använda **if**-satser. Till exempel i denna kod:

```
def elem_access(xs,i):
    try:
        if i >= len(xs):
            raise IndexError
    except IndexError:
        print("Index is out of range.")
    return xs[i]
```

Det är ju ingen poäng med att först skapa ett särfall och sedan genast fånga det. Ett annat misstag i just detta exempel är att om det uppstår ett fel, så *fortsätter* programmet efter en felutskrift, här med att försöka komma åt element i trots att i är för stort. Om man fångar ett särfall så ska man hantera det på

något sätt. Det är ofta rimligt att avsluta programmet med ett felmeddelande och `quit()`. I det här fallet, när `elem_access` verkar vara en hjälpfunktion, är det bättre att bara skapa ett särfall vid problem och låta användaren av hjälpfunktionen bestämma vad man ska göra.

```
def elem_access(xs,i):    # Bättre version
    if i >= len(xs):
        raise IndexError
    return xs[i]
```

5.1.6.2 Överanvändning

Använd inte särfall för att tackla att kod inte fungerar som du hade tänkt dig.

I exemplet nedan skapas en ny uppslagstabell `d` för nycklar `e`, tagna från en lista, genom att anropa en funktion som använder informationen i uppslagstabellen `data`. Tydligt har programmeraren märkt att alla värden från `selected` inte finns med i `data` och därför skyddat sig genom att fånga de `KeyError`'s som kan uppstå:

```
data = {1 : "Make bed",
        2 : "Shower",
        4 : "Eat breakfast"}
selected = [1,3,2]

d = {}
for elem in selected:
    try:
        d[elem] = perform_task(data[elem])
    except KeyError:
        print()          # Bad line of code!
```

Det är bättre att använda en `if`-sats för att undvika felen:

```
data = {1 : "Make bed",
        2 : "Shower",
        4 : "Eat breakfast"}
selected = [1,3,2]

d = {}
for elem in selected:
    if elem in data:
        d[elem] = perform_task(data[elem])
```

5.2 Listomfattningar

Listomfattning (eng: *list comprehension*) är en teknik för att skapa listor som också är populär bland funktionella programmeringsspråk och därför ofta tas upp i det sammanhanget. Listomfattning är praktiskt för att initiera listor med värden att jobba vidare med. En fördel, som kanske ligger bakom listomfattningars popularitet, är att de kan vara mycket tydliga tack vare sin nära koppling till matematisk notation. Betrakta till exempel mängden av kvadrater av tal upp till 10:

```
[x**2 for x in range(10)]
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Man kan också filtrera bort elementen — inte alla behöver presenteras. Till exempel alla udda heltal upp till 20 kan skrivas så här:

```
[x for x in range(20) if x % 2 == 1]
```

```
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

(Man kan korta ner det uttrycket lite till, kan du se hur?)

Vi kan även anropa på funktioner som konverterar element enligt något system

```
def transform(x):  
    if x % 2 == 0:  
        return "Even"  
    if x % 3 == 0:  
        return True  
    else:  
        return x  
print([transform(x) for x in range(20)])
```

```
['Even', 1, 'Even', True, 'Even', 5, 'Even', 7, 'Even', True]
```

Sammanfattning: en listomfattning har tre delar:

- resultatdelen, med ett uttryck som ger elementen i resultatet,
- generatordelen, där **for** används för att plocka fram värden till resultatdelen, och
- predikatdelen, som bestämmer vilka element som ska presenteras.

Mer komplicerat exempel med nästlad loop:

```
[(a,b) for a in range(2,8) for b in range(2,8) if a % b == 0]
```

```
[(2, 2), (3, 3), (4, 2), (4, 4), (5, 5), (6, 2), (6, 3), (6, 6), (7, 7)]
```

Smidigt för att få alla kombinationer av element i två listor. Notera att utdata är en lista av **tupler**, dvs par av element.

En användbar funktion som tar två listor med samma längd och gör en lista av tupler där man parat ihop elementen på samma index med varandra är `zip` (finns dock redan implementerat i python). Man skriver den lätt med en listomfattning.

```
def zip(xs,ys):  
    if len(xs) != len(ys):  
        print("zip unequal lengths!")  
        return []  
    return [ (xs[i],ys[i]) for i in range(len(xs)) ]
```

5.2.1 dict-omfattningar

Listomfattning är den traditionella tekniken, men i Python har man generaliserat dessa till uppslagstabeller (och mängder som kommer i senare kapitel). Man kan alltså skapa uppslagstabeller med dict-

omfattning (eng. *dict-comprehensions*). Man måste i dessa fall se till att man skapar både nycklar och värden:

```
my_dict = {x: x ** 2 for x in (2, 4, 6)}  
print(my_dict)
```

```
{2: 4, 4: 16, 6: 36}
```

```
squares = {str(x) + '^2': x ** 2 for x in range(4)}  
print(squares)
```

```
{'0^2': 0, '1^2': 1, '2^2': 4, '3^2': 9}
```

```
lst = ['1', '42', '13']  
my_dict2 = {'key_' + x: int(x) for x in lst}  
print(my_dict2)
```

```
{'key_1': 1, 'key_42': 42, 'key_13': 13}
```

5.3 Felsökning

När man programmerar drabbas man av misstag, missförstånd, och felbedömningar. Det skapar fel i koden, som ibland kallas för “buggar” och är en försvenskning av det engelska ordet “bug” som betyder insekt, men har också använts som benämning på fel i tekniska system redan innan datorer började byggas. Buggar kan få program att krascha, det vill säga att programmet avbryts abrupt eftersom det inte går att fortsätta, eller bara orsaka ett felaktigt resultat som kanske inte ens märks. Man ska därför vara glad över felmeddelanden: det är ju betydligt värre med fel som man inte är medveten om.

Vi kan dela in programmeringsfel som språkliga misstag och körfel (“runtime errors”). Språkliga misstag fångas av kompilatorn och är oftast lätta att förstå när man har lärt sig programmeringsspråket. Det kan vara svårare med körfelen, eftersom man ofta upptäcker symptom, det vill säga konsekvenser av misstag eller problem. Det felmeddelande man får talar alltså inte alltid om vilket det egentliga felet är, utan är ett symptom på det egentliga felet. Som exempel: om ditt program kraschar på grund av att en uppslagstabell saknar den nyckel som används för en uppslagning, så är kanske uppslagningen i sig inte felet, utan det kan vara att fel nyckel används eller så har nyckeln inte har lagts in. Det är sällan rätt att “bota” symptom på fel när man programmerar, utan det är viktigt att lösa det underliggande problemet. I exemplet med uppslagstabeller är det därför inte en lösning att lägga till kod så att inget händer om en nyckel inte hittas i uppslagstabellen, utan man bör istället fastställa varför nyckeln används eller varför nyckeln inte finns inlagd.

5.3.1 Exempel på fel

Fel kommer ofta av missförstånd. Man kanske missförstår en detalj i en algoritm eller har en felaktig idé om hur indata faktiskt ser ut. Ibland har man missförstått det programmeringsspråk man arbetar i och det är lätt hänt eftersom det finns många detaljer att hålla reda på.

Vi kan titta på några exempel på återkommande fel, för att visa vad man kan behöva vara uppmärksam på.

5.3.1.1 Exempel: problem vid sortering

Antag att du behöver sortera en lista med tal och experimenterar med den här snutten kod:

```
lst = [10, 3, 4]
lst = lst.sort()
for elem in lst:
    print(elem)
```

Om man kör det här lilla programmet får man ett felmeddelande vid for-loopen: `TypeError: 'NoneType' object is not iterable`. Varför det? Det är ju uppenbart att det finns element i listan? Felet uppstår på grund av ett missförstånd om hur `sort` fungerar. Uttrycket `lst.sort()` returnerar `None`, men ändrar ordningen på elementen i `lst`, och då blir ju rad två i koden olycklig eftersom `lst` tilldelas `None`.

Lägg märke till att felet görs på rad två, men detekteras på rad tre. Det här är vanligt: när vi observerar fel så är det ofta symptom på fel som gjorts tidigare.

Ett sätt rätta till felet ovan är att byta till funktionen `sorted`:

```
lst = [10, 3, 4]
lst = sorted(lst)
for elem in lst:
    print(elem)
```

Detta fungerar eftersom `sorted` returnerar en ny kopia av listan, men sorterad.

5.3.1.2 Exempel: dåligt returvärde

En av våra tumregler är att man alltid ska returnera ett resultat från en funktion. Man ska ha goda skäl för att inte returnera något. Anledningen är att undvika ett vanligt nybörjarmisstag, som även rutinerade programmerare kan göra.

Antag att vi vill ha en liten funktion som tar fram de tre största värdena ur en lista. Vi provar oss fram och konstaterar att det löses enkelt med två rader kod:

```
lst = [10,3,4,5,1,2]
lst = sorted(lst)
print(lst[-3:])
```

```
[4, 5, 10]
```

Bra, då gör vi det till en funktion:

```
def largest_three(lst):
    lst = sorted(lst)
    print(lst[-3:])
```

Funktionen testas vi, som man bör, men då blir det fel.

```
a, b, c = largest_three([10,3,4,5,1,2])
```

```
[4, 5, 10]
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: cannot unpack non-iterable NoneType object
```

Resultatet skrivs ut, men tilldelningen av returvärdet fungerar inte, eftersom det inte finns en retur-sats, vilket ger returvärdet `None`. Man ska alltid returnera de intressanta värdena!

```
def largest_three(lst):  
    lst = sorted(lst)  
    return lst[-3:]
```

5.3.1.3 Exempel: iteratorer är inte listor

Inspirerat av lösningen ovan kanske man tänker sig en liknande uppgift, löst på liknande sätt. Antag att vi vill skriva ut en lista i omvänd ordning och se visa maxvärdet.

```
lst = [10, 3, 4]  
lst = reversed(lst)  
for elem in lst:  
    print(elem)  
print('Largest:', max(lst))
```

Nu blir det fel igen, på sista raden: `ValueError: max() arg is an empty sequence`. Varför det? Det blir inget fel om man använder `sorted` istället för `reversed`. Nej, men de två funktionerna har olika typer på sina returvärden. Med `sorted` får vi en lista och den kan man utan problem använda upprepade gånger, men `reversed` returnerar en iterator och dessa används en och endast en gång. När `for`-loopen är klar så är iteratorn ”förbrukad” och `max` har inget att arbeta med.

Det här är ett ganska vanligt misstag. Iteratorer används flitigt i Python, vilket är bra eftersom de är effektiva och minnessnåla, men man får vara uppmärksam på att en iterator behöver omvandlas till en lista ibland.

Hur rättar du felet?

5.3.2 Förstå felmeddelandet

Ett första steg till att bli bra på felsökning är att läsa felmeddelanden noga. Om programmet avslutar med ett fel så brukar felmeddelandet tala om ganska tydligt varför det avslutar. Återigen, detta är ofta bara symptom på det egentliga felet, men symptom är viktiga.

Felmeddelanden i Python har alltid samma struktur.

- Först visas var felet uppstår. Det görs genom att visa ett *stack trace* som anger vilka funktionsanrop som är involverade och i vilka filer och på vilka rader de anropas.
- Sist visas var felet uppstår, med filnamn och rad, och en enkel beskrivning av felet.

Betrakta den här konstiga koden, skriven enbart för att visa upp ett felmeddelande:

```
def f1(x):  
    return f2(x)  
  
def f2(x):  
    return f3(x)  
  
def f3(x):  
    return 17 / x  
  
f1(0)
```

Om man kör programmet anropas alltså $f1(0)$, som anropar $f2(0)$, som i sin tur anropar $f3(0)$ där division med noll uppstår. Det resulterar i det här felmeddelandet (åtminstone i Python 3.9):

```
Traceback (most recent call last):
  File "~/src/kod.py.py", line 11, in <module>
    f1(0)
  File "~/src/kod.py.py", line 3, in f1
    return f2(x)
  File "~/src/kod.py.py", line 6, in f2
    return f3(x)
  File "~/src/kod.py.py", line 9, in f3
    return 17 / x
ZeroDivisionError: division by zero
```

Ganska mycket text för ett enkelt fel, men det är illustrativt. Lägg särskilt märke till att det framgår på vilken rad det blir fel, här rad 9, och att det framgår att symptomet är just division med noll. Editorer som Spyder visar hjälpsamt just den problematiska filen och raden. Och tack vare "Traceback", som ger oss vårt *stack trace*, kan man utläsa hur funktionerna anropat varandra, vilket är ytterligare tips inför felsökningen.

En del felmeddelanden kan vara svåra att läsa, och för en nybörjare så kan viss terminologi vara alltför teknisk, men även om man inte förstår allt så kan man ändå hitta värdefulla ledtrådar.

5.3.3 Leta fel med spårutskrifter

Spårutskrifter är en beprövad och välanvänd teknik för att leta fel. Idén är enkel: lägg in `print`-satser som avslöjar värden på variabler och visar både hur variabelvärden ändras och hur programflödet går. Man kan se det som ett noggrant sätt att leta symptom som kan avslöja vad felet är.

Som exempel kan vi titta på exemplet om symbolisk derivering av polynom från avsnittet om Datarepresentation, men med ett fel insmuget:

```
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial:    # Fel!
        new_coefficient = term * degree
        result.append(new_coefficient)
        degree += 1
    return result
```

Antag att vi har observerat att det blir något fel utan att lyckas lista ut vad det beror på. En enkel spårutskrift visar de relevanta variabelvärden i varje iteration:

```
def derivative(polynomial):
    result = []
    degree = 1
    print('degree term new_coefficient')    # Praktisk rubrik
    for term in polynomial:    # Fel!
        new_coefficient = term * degree
        print(degree, term, new_coefficient) # Spårutskrift
        result.append(new_coefficient)
```

```
degree += 1
return result
```

Denna spårutskrift skulle förhoppningsvis snabbt visa upp att vi av misstag fått med konstanttermen i deriveringen.

När man jobbar med spårutskrifter händer det lätt att man känner ett behov av att krydda med förklarande information. Här ovan var det till exempel bra att lägga in en rubrikrad. Behovet är så pass vanligt att Python har en särskild finess för spårutskrifter lite gömd i f-strängar: om du skriver `f'{x}'` så ersätts `{x}` av värdet på variabeln `x`, men om du skriver `f'{x=}'` så sätts istället `x=` och värdet på variabeln in. Så exemplet ovan kan bättre skrivas så här:

```
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial:    # Fel!
        new_coefficient = term * degree
        print(f'{degree=}, {term=}, {new_coefficient=}') # Spårutskrift
        result.append(new_coefficient)
        degree += 1
    return result
```

Prova! Om vi provkör detta med polynomet $1 + x + x^2$ så ser det ut så här:

```
derivative([1,1,1])
```

```
degree=1 term=1, new_coefficient=1
degree=2 term=1, new_coefficient=2
degree=3 term=1, new_coefficient=3
```

Här ska man se att de finns en term med grad 3, vilket är ju är fel eftersom vi har ett andragradspolynom som indata.

Spårutskrifter är bekvämt och praktiskt att använda, men betraktas ofta som dålig praxis. Nackdelarna med spårutskrifter är att det är arbetsamt och ibland onödigt att lägga in dem, man måste ju oftast ta bort dem också, men de används ändå vitt och brett av erfarna programmerare. I scriptspråk som Python är kostnaden med spårutskrifter ändå ganska låg eftersom man tenderar att arbeta ganska interaktivt med koden, mer än i vissa andra språk, och det är lätt att testa kod.

5.3.3.1 Loggning

Om man finner att spårutskrifterna är användbara, så kanske man entusiastiskt börjar planera för särskilda funktioner för spårutskrifter, avstängbara, där man kan få dem skrivna till en separat fil för spårutskrifter. I så fall har man börjat med loggning och det finns ingen anledning till att skriva egen kod för det. Det är ett så vanligt behov att det finns en standardmodul, med namnet `logging`, för ändamålet. Syftet med modulen är att göra det lätt att logga status på program, inte bara för att leta fel, utan mer allmänt för att hålla koll på vad ett system håller på med. Många datorsystem arbetar över lång tid med viktiga systemtjänster och då blir loggar ett viktigt verktyg för att hålla koll på vad som gjorts i ett system.

Här är ett exempel på hur deriveringskoden kan justeras till att logga status till filen `derivative.log`:


```

import logging
logging.basicConfig(filename='derivative.log', level=logging.DEBUG)
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial:
        new_coefficient = term * degree
        logging.debug(f'{degree=}, {term=}, {new_coefficient=}') # Spårutskrift
        result.append(new_coefficient)
        degree += 1
    return result

```

De två första raderna initierar loggningen och man kan se vilken fil som loggmeddelanden hamnar i och vilken *nivå* som ska användas. Utan `level=logging.DEBUG` kommer debug-meddelanden och att vara utan effekt, men utskrifter gjorda med `logging.warning` eller `logging.error`, två andra utskriftsmetoder med högre prioritet kommer däremot skrivas ut.

Om man evaluerar koden ovan och sen `derivative([1, 1, 1])` kommer man återfinna dessa utskrifter i loggfilen:

```

DEBUG:root:degree=1, term=1, new_coefficient=1
DEBUG:root:degree=2, term=1, new_coefficient=2
DEBUG:root:degree=3, term=1, new_coefficient=3

```

I vänsterkanten anges vilket sorts loggmeddelande som getts ("DEBUG") och termen "root" som anger källan till meddelandet, vilket i vårt exempel inte har någon funktion. Modulen stödjer mer avancerad användning än vad vi visar här.

5.3.4 Leta fel med debugger

Programmerarens kanske bästa hjälpmedel är *debuggern*, på svenska även känd som *avlusare*. Det är ett program som används för att köra ditt program på ett kontrollerat sätt, med möjlighet att stega sig fram, rad för rad, och granska variablers värden utan spårutskrifter.

I editorer särskilt utformade för Python, som Spyder och PyCharm, är debuggern inbyggd. Andra editorer, till exempel VS Code, har tillägg som erbjuder stöd för debugger. I vissa verktyg finns kontrollerna för debuggern alltid tillgängliga, och i andra måste du särskilt instruera editorn att "nu vill jag debugga". Oavsett vilken editor och debugger som du använder så kan du räkna med att det finns flera återkommande och användbara finesser, och som dessutom återfinns för de flesta programmeringsspråk. Vi ska titta på några av de grundläggande finesserna.

5.3.4.1 Brytpunkter

En brytpunkt markeras ofta med en röd symbol i kodens vänsterkant och den betyder helt enkelt "stanna innan denna rad exekveras". Du kan alltså starta ditt program med debuggern och när körningen kommer till brytpunkten så pausar den. Körningen avbryts inte helt, utan kan återupptas när man är redo för det.

I pausen har du möjlighet att granska vilka värden som de olika variablerna har, vilket förhoppningsvis ger dig ledtrådar om felet. Att arbeta med brytpunkter och inspektion av variabler är förstås ganska likt att använda spårutskrifter, men fördelen är att du inte behöver ta bort spårutskrifterna när du är klar. Om ditt program dessutom har många variabler med lite komplicerade datastrukturer så brukar

brytpunkter vara enklare att arbeta med. En fördel är att du kan undersöka datastrukturer och behöver inte i förväg välja ut vad som ska visas.

5.3.4.2 Stegning

Brytpunkter ger dig ögonblicksbilder av programmet och det är inte alltid tillräckligt för att förklara vad som går fel. Därför är det praktiskt att kunna stega sig fram i programmet, rad för rad, och kontinuerligt inspektera variabler om så behövs eller helt enkelt bara se vilken väg i programmet man råkar ta.

Det finns två sätt att stega och du bör vara beredd att använda båda. I det enkla steget, ofta kallat *step* i en debugger och tillgängligt via en knapp, så är det alltid aktuell rad som exekveras. Om raden innehåller ett funktionsanrop så följer man koden in i funktionen, vilket ibland innebär att man följer körningen in i Pythons inre delar. Det kan vara både förvirrande och tidsödande. Det är sällan man behöver granska Pythons egen implementation för att förstå varför ens program gör fel. I sådana tillfällen är det bra att använda det andra sättet att stega, ofta kallat *next* eller *step over*. Med *next* så instruerar man debuggern att exekvera kodrader och fortsätta till nästa rad. Man hoppar alltså över detaljerna i den funktion som eventuellt anropas.

Det brukar också finnas möjlighet att säga "fortsätt till sista raden på funktionen" (i Spyder: "step return"), som är ett bra sätt att komma framåt på ett snabbare men fortfarande kontrollerat sätt.

5.3.5 Strategier för felsökning

Även om man behärskar tekniker för felsökning så kan det vara svårt eller långsamt att hitta det verkliga misstaget. Det är värt att tänka igenom hur man kan avslöja det underliggande felet.

I avsnittet Defensiv programmering ges tips om hur man kan använda `assert` för att lägga in villkor i sitt program. Villkoren ska beskriva utsagor som borde vara sanna där `assert`-raderna ligger. Om villkoret gjorts falskt av en bug så avbryts programmet.

Du kan också läsa om enhetstestning, som är ett sätt att systematiskt och automatiserat testa att programmets funktioner är korrekta. Om alla funktioner i ett program har enhetstester så får man hjälp att tidigt hitta misstag.

Med eller utan enhetstestning, det kommer alltid vara viktigt att använda bra strategier för att hitta sina fel. Vi ska nu gå igenom några strategier.

5.3.5.1 Avgränsa fel

Om ett program kraschar på rad 917 efter att ha exekverat ett flertal olika funktioner så kan misstaget ligga på just den raden eller raden innan, men man ska inte utgå ifrån att felet är nära rad 917. Fel kan propagera i flera led utan att man lägger märke till det. Det blir då viktigt att kunna lyfta blicken och titta längre tillbaka i programmet. Att då granska först rad 917, sen rad 916, och rad 915, och så vidare längre bakåt, är långsamt. Det är mer effektivt att, om felet inte ligger nära rad 917, försöka göra avgränsningar. Är indata till den senast anropade funktionen korrekta? Hur skapades dessa indata?

När felet är riktigt förvirrande svårt att hitta kan man behöva skapa "kontrollpunkter" i sitt program för att verifiera att beräkningarna är korrekta fram till dem. På så sätt kan man avgränsa och isolera källan till felet.

När man har ett fungerande program som man bygger ut eller förändrar så är förstås den nya koden huvudmisstänkt och ska, oavsett var de ligger, granskas först.

5.3.5.2 Skriv om

En del fel är alldeles framför ögonen, men man ser dem inte pga felaktiga antaganden eller bara ren förvirring. Förvirring och tillfällig blindhet för vissa misstag är att vänta, eftersom man ofta måste hålla många saker i huvudet när man programmerar.

En typ av misstag som uppstår lättare i scriptspråk (i motsats till språk som Java där man måste deklarera sina variabler) är felstavning. I förenklad form kan det se ut så här:

```
identifier = 7           # A first assignment
idenfifier = identifier * 2 # Updating the assignment
print(identifier)
```

Varför skrivs värdet 7 ut? Ser du misstaget?

Om man kan lokalisera ett fel till en funktion, men ändå inte lyckas identifiera felet, så är en strategi att formulera om funktionen. Bara processen att skriva en funktion på nytt, kanske med andra identifierare eller med en lite annorlunda algoritm, kan bota den tillfälliga blindheten mot enkla misstag som stavfel, eller avslöja underliggande antaganden som man inte först reagerat på.

5.3.5.3 Granska indata

En del problem beror på att indata innehåller konstigheter som det kan vara svårt att lägga märke till. Moderna standarder har botat det här i hög grad, men man kan fortfarande stöta på problem som till exempel beror på att indata har skapats på en Mac men ska analyseras under Windows.

För det mesta kan man förlita sig på att textfiler använder teckenkodningen UTF-8, som är kompatibelt med äldre teckenkodningar som ASCII. Om man jobbar i äldre versioner av Python, version 2.7 och tidigare, så kan man inte förlita sig på att programmet automatiskt klarar UTF-8, och då kan till exempel svenska tecken bli konstiga. Teckenkodning ger alltså sällan problem.

Det är däremot lätt hänt att man gör antaganden om formatet på indata. Om mellanrum är viktigt, kodas det med ett blanksteg ("space") eller med ett TAB-tecken? En sån detalj är lätt att missa även när man tittar på indatafilen i en texteditor. Det kan därför hjälpa att skriva kod som hanterar både blanksteg och TAB-tecken.

Det är mycket vanligt att man flyttar filer mellan datorsystem, och då kan man upptäcka att radbrytningar hanteras olika på olika datorsystem. På Mac och Linux brukar ny rad kodas med ett tecken, '\n' i Python, men på en PC så används '\n\r', vilket läses som "new line and carriage return". På PC ser du alltså spår av de första datorskrivarna och deras koppling till mekaniska skrivmaskiner: först ska man mata fram papper till en ny rad, och sen ska skrivhuvudet flyttas längst till vänster.

Ett sätt att slippa bry sig om dessa detaljer är att förlita sig på etablerat kod för att läsa data, och det kan du läsa om i ett senare kapitel.

5.3.5.4 Välj dina testdata

När man letar fel så ska man välja sina testdata med omsorg.

- Arbeta med små testdata. Om ditt mål att analysera en stor datafil så bör du skapa mindre filer med liknande egenskaper för testning. Det snabbar upp felsökningen.
- Fel uppstår ofta på mer ovanliga indata, och därför är det bra att testa på specialfall som man konstruerar. Om du läser data från fil så bör din kod kunna läsa en tom fil utan att krascha. En funktion som arbetar med strängar bör klara den tomma strängen, '', utan problem. Om du

implementerar en numerisk funktion $f(x)$ så bör den fungera på $x = 0$ (om funktionen är definerad där).

Tänk också på att du antagligen kommer att behöva testa många gånger, och kanske även i framtiden. Det är därför bra konstruera testdata och spara dem för framtida behov.

5.4 Uppgifter

1. Vad händer om vi anropar funktionen `divide_by_elems` (funktionen given i avsnitt 5.1.2.3) med en sträng som argument för `x`? Skriv om funktionen så att om strängen kan göras om till ett heltal ska den fortsätta utan att krascha, annars ska den returnera `None`. Lös detta i funktionen med hjälp av `try` och `except`.
2. Skriv om funktionen `divide_by_elems` så att den inte bryter efter första fel, utan fortsätter att dela talet `x` med rader som kommer efter. Till exempel för filen `numbers.txt` som ovan ska funktionen returnera listan (och eventuellt skriva ut diverse meddelanden från felhanteringen):

```
[2.0, 1.0, 0.5, 0.6666666666666666]
```

3. Givet följande lista

```
xs = [2, -167, 12, 5676, -14, -7, 12, 12, -1]
```

Skapa med hjälp av en listomfattning listan `ys` som innehåller endast de positiva talen i `xs`.

4. Skapa med listomfattning en lista `ys` där alla talen i listan `xs` i uppgift 3 har konverterats till positiva om de är negativa (t.ex. `-167` blir `167`).
5. Givet funktionen nedan

```
def is_prime(x):  
    if x >= 2:  
        for y in range(2,x):  
            if not ( x % y ):   
                return False  
    else:  
        return False  
    return True
```

Skapa en lista med alla primtal under 100 genom att använda en listomfattning.

6. Givet följande lista

```
xs = [2, 3, 5, 7]
```

Använd listomfattning för att skapa en lista `ys` som innehåller produkterna av alla par av element från `xs`, dvs elementen `2 · 2`, `2 · 3`, `2 · 5`, `2 · 7`, `3 · 2`, osv.

7. Skapa med listomfattning en lista `ys` som innehåller produkter från multiplikation av alla parvisa element i `xs` från uppgift 6 utom elementen själva.

Obs: Talet 2 på index 0 i `xs` ska inte tolkas samma som 2 på index 1. Du kan alltså inte testa om talen är lika utan måste testa om index är samma.

Tips: använd `range`.

8. Skriv om labb 1 med exceptions så att programmet inte kraschar om man skriver in något fel.